



TD 02 - Configurer une DB

Ce TD vous guidera à travers les étapes nécessaires pour développer, configurer, et déployer une application Spring Boot connectée à une base de données sur la plateforme Alwaysdata.

Objectifs du TD

À l'issue de ce TD, vous serez capable de :

1. Créer une base de données sur AlwaysData.
2. Configurer la connexion à la base de données à l'aide de variables d'environnement pour garantir la sécurité et la flexibilité.
3. Déployer l'application Spring Boot sur AlwaysData et la lier à une base de données en ligne.
4. Rédiger un script shell permettant d'automatiser le processus complet de déploiement.

PRÉ-REQUIS

1. Connaissances de base en Spring Boot, en gestion de bases de données relationnelles et en script shell.
2. Environnement de travail configuré avec Java, Git et un IDE de votre choix.
3. Un compte Alwaysdata associé à votre machine via une clé SSH.

Un service rest avec une base de données

Commencez par récupérer l'application Spring-Boot à déployer.

```
git clone https://git.esi-bru.be/4dop1dr-ressources/demo.git
```

Cette application est composée du contrôleur suivant :

```
1 @RestController
2 public class ServiceRestController {
3
4     @Autowired
5     private PersonDB repository;
```

```
6
7     @Autowired
8     private Environment environment;
9
10    @GetMapping("/config")
11    public Map<String, String> getEnvironmentDetails() {
12        Map<String, String> envDetails = new HashMap<>();
13
14        // Récupération des propriétés de configuration Spring Boot
15        // depuis application.properties
16        envDetails.put("Application Name",
environment.getProperty("spring.application.name"));
17        envDetails.put("Port Number",
environment.getProperty("server.port"));
18
19        // Récupération des propriétés système
20        envDetails.put("Java Version",
System.getProperty("java.version"));
21        envDetails.put("Operating System", System.getProperty("os.name"));
22        envDetails.put("User Name", System.getProperty("user.name"));
23
24        //Récupération des données de La Personne d'identifiant 1
25        Person person = repository.findById(1).get();
26        // Ajouter Les valeurs provenant de La base de données
27        envDetails.put("Person ", person.toString());
28
29        return envDetails;
30    }
31
32 }
```

Le contrôleur de l'application Spring expose un service REST qui retourne en plus des détails sur la configuration de l'environnement dans lequel l'application s'exécute, des données relatives à une personne persistées dans une base de données.

Lors du démarrage du serveur Tomcat, un serveur de bases de données embarqués de type H2 est démarré. Le fichier `application.properties` contient les paramètres de connexion vers cette base de données

`application.properties`

```
spring.application.name=demo pour devops
server.port=8080
```

```
#Configuration de la base de données embarquée
spring.datasource.url=jdbc:h2:mem:mydatabase
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.defer-datasource-initialization=true
```

EXERCICE

1. Démarrez l'application via `mvn spring-boot:run`.
2. Vérifiez que le service web est fonctionnel. Vous devriez recevoir une personne en plus des propriétés.
3. Connectez vous à la console de la base de données via localhost:8080/h2-console
4. Entrez `sa` comme nom d'utilisateur
5. N'entrez aucun mot de passe.
6. Vérifiez que la table `person` a été créée.

Pensez-vous que cette configuration sans mot de passe est correcte pour déployer l'application ?

MODE DÉVELOPPEMENT VS MODE DÉPLOIEMENT

Via la commande `mvn package` un *jar* est créé en mode **déploiement**, pas en mode **développement**; il ne contient pas de console h2. Pour voir la console h2, exécutez l'application en mode développement via un bouton de votre IDE ou via la commande `mvn spring-boot:run`.

Préparer l'infrastructure

Si vous utilisez le serveur de base de données H2 embarqué avec votre application, les données sont stockées dans la mémoire vive de l'application et **disparaissent** à chaque arrêt ou redémarrage. Ainsi, sur Alwaysdata, toute interruption de l'application entraînerait la perte des données, car H2 ne permet pas leur persistance. Une meilleure pratique consiste à créer une base de données directement sur Alwaysdata via leur interface, puis à configurer votre application pour utiliser cette base de données externe à la place de la base de données H2 embarquée.

Créer la base de données

Sur la page de votre compte Alwaysdata, allez dans le menu `Databases > MySQL`. Appuyez sur `Add a database` et intitulez cette base de données `g12345_demo_devops`. N'oubliez pas de sélectionner les permissions adéquates pour votre utilisateur, c'est à dire `all rights`.

Sur la page listant vos bases de données MySQL, vous trouverez l'information `MySQL host: mysql-g12345.alwaysdata.net`, qui vous permettra de configurer les paramètres de connexion pour votre application déployée. Cette même page fournit un lien vers la console phpMyAdmin : <https://phpmyadmin.alwaysdata.com/>.

EXERCICE

Connectez-vous à la console phpMyAdmin en utilisant votre nom d'utilisateur (`g12345`) et le mot de passe de votre compte AlwaysData, puis vérifiez que la base de données `g12345_demo_devops` est **vide**.

Les variables d'environnements

Sur la machine locale

Sur votre machine, modifiez le fichier `application.properties` et supprimez (ou mettez en commentaire) les informations concernant H2.

`application.properties`

```
1 spring.application.name=demo pour devops
2 server.port=8080
```

Dans un terminal *bash*, exportez les variables d'environnements systèmes comme ci-dessous :

déclaration des variables dans le terminal

```
export SPRING_DATASOURCE_URL=jdbc:h2:mem:mydatabase
export SPRING_DATASOURCE_USERNAME=sa
export SPRING_DATASOURCE_PASSWORD=
export SPRING_JPA_DEFER_DATASOURCE_INITIALIZATION=true
```

Suite à ces exports, les variables supprimées du fichier `application.properties` sont définies dans votre terminal.

Dans **le même terminal**, empaquetez à présent votre application, démarrez-la et consommez le service REST afin de **vérifier** que l'application fonctionne suite à ces changements.

Sur la machine distante

Connectez-vous à votre compte Alwaysdata et consultez la page des sites. Modifiez les propriétés de votre site () et ajoutez dans la section `Environment` les variables d'environnements systèmes ci-dessous.



N'oubliez pas de remplacer g12345 par votre login dans les variables ci-dessous.

déclaration des variables sur le serveur AlwaysData

```
SPRING_DATASOURCE_URL=jdbc:mysql://mysql-  
g12345.alwaysdata.net:3306/g12345_demo_devops  
SPRING_DATASOURCE_USERNAME=g12345  
SPRING_DATASOURCE_PASSWORD=MOT DE PASSE DE VOTRE COMPTE ALWAYSDATA  
SPRING_JPA_DEFER_DATASOURCE_INITIALIZATION=true  
SPRING_JPA_HIBERNATE_DDL_AUTO=create  
SPRING_SQL_INIT_MODE=always
```

Il s'agit de la même opération que vous avez effectuée sur la machine locale. Deux différences essentielles sont à noter :

1. L'absence du mot clé `export`
2. Les variables sont adaptées pour une base de données MySQL.

Finalement :

1. Enregistrez vos modifications.
2. Copiez le fichier JAR de votre application. via la commande `scp` dans le dossier `www`.
3. Consommez le service via l'url `http://g12345.alwaysdata.net/config`.
4. Connectez-vous à la console phpMyAdmin : <https://phpmyadmin.alwaysdata.com/>.
5. Vérifiez que la table `person` est créée et contient 2 records.

MOT DE PASSE VISIBLE DANS LES VARIABLES D'ENVIRONNEMENT

Votre mot de passe est affiché en clair et reste visible dès que la page des paramètres de votre site est ouverte. Dans les prochains TD, nous introduirons le concept de variables secrètes pour remédier à ce problème.

La logique DevOps : l'automatisation

EXERCICE

Afin de prendre conscience de l'avantage d'automatiser des processus répétitif et de se placer dans une logique DevOps professionnelle écrivez un script shell qui :

1. Compile l'application Spring Boot via la commande `mvn compile`.
2. Exporte les variables d'environnements locales sur votre système d'exploitation.
3. Lance les tests unitaires de l'application via la commande `mvn test`.
4. Empaquete l'application via la commande `mvn package`.
5. Déploie l'application sur Alwaysdata à l'aide de la commande `scp`.
6. Consomme le service rest.

Le script doit vérifier après chaque action le succès ou l'échec de cette action et arrêter le script en indiquant à l'utilisateur la cause d'arrêt en cas d'erreur.

Pensez à modifier la variable `spring.application.name` à chaque déploiement pour confirmer que la mise à jour a bien été prise en compte. Par exemple, vous pouvez utiliser une valeur unique basée sur le numéro de l'essai :

```
spring.application.name=demo pour devops - déploiement numéro 42
```

Cela permet de vérifier facilement que l'application déployée correspond à la version la plus récente.

Vous pouvez également dans le menu du site de Alwaysdata appuyer sur le bouton affichant les `logs` () de votre application pour consulter les messages d'erreurs.

A titre d'exemple voici un script shell automatisant l'étape d'empaquetage de notre application. Remarquez l'utilisation de la condition `if [ $? -ne 0 ]` permettant de détecter si la commande précédente s'est terminée en erreur ou en succès.

```
1  #!/bin/bash
2
3  # Chemin vers votre projet à compléter
4  WORK_DIR="/home/..."
5
6  cd "$WORK_DIR"
7
8  echo "  Création du fichier JAR..."
9  ./mvnw package
10 if [ $? -ne 0 ]; then
11     echo "  Erreur : Échec de la création du fichier JAR."
12     exit 1
13 fi
14 echo "  Fichier JAR créé : $JAR_FILE."
15
```